

Predicting The Severity of Traffic Accidents

Big Data — Final Project
Innopolis University

Team 31

Anna Belyakova	<code>a.belyakova@innopolis.university</code>
Sofia Pushkareva	<code>s.pushkareva@innopolis.university</code>
Daria Alexandrova	<code>d.alexandrova@innopolis.university</code>
Aisylu Fattakhova	<code>a.fattakhova@innopolis.university</code>

May 9, 2026

Contents

1	Introduction	2
2	Data Description	2
3	Architecture of Data Pipeline	4
4	Data Preparation	5
5	Data Analysis	8
6	Modeling and Prediction	12
7	Data presentation	14
8	Conclusion	17
9	Reflections on own work	17
	The table of contributions of each team member	18
10	References	18

1 Introduction

Business objectives

Traffic accidents pose a significant challenge to public safety, resulting in loss of life, severe injuries, and substantial economic costs due to property damage and traffic congestion. The primary business objective of this project is to develop a robust big data pipeline and a predictive machine learning model capable of classifying the severity of traffic accidents based on real-time features such as weather conditions, time of day, and road environment.

By accurately predicting accident severity immediately upon occurrence using the nationwide US Accidents dataset, several key stakeholders can derive actionable insights:

- **Emergency Medical Services (EMS):** Predicting the potential severity of an incident allows dispatchers to optimize resource allocation, minimizing response times.
- **Department of Transportation (DoT):** Analyzing the correlation between accident severity and various factors helps authorities identify high-risk zones for targeted infrastructure improvements.
- **Navigation Services:** Systems like Google Maps can estimate road clearance times more accurately and proactively reroute drivers.

2 Data Description

Dataset Overview

The dataset used in this project is titled "**US Accidents (2016–2023)**" and was obtained from Kaggle. It is a countrywide traffic accident dataset covering 49 states of the United States. The dataset contains approximately 7.7 million accident records collected from February 2016 to March 2023. The uncompressed CSV file size is massive, requiring distributed big data tools for processing. To ensure efficient processing and balanced analysis, data was chunked and cleaned, and categorical/boolean variables were standardized.

Dataset Source and Purpose

The original data was collected using multiple APIs that provide streaming traffic incident (or event) data. These APIs broadcast traffic data captured by various entities, including the US and state departments of transportation, law enforcement agencies, traffic cameras, and traffic sensors within the road networks.

Traffic accidents cause significant economic loss, property damage, and delays. This dataset enables the analysis of environmental, spatial, and temporal factors leading to accidents, providing valuable insights into predictive modeling opportunities for emergency responders and navigation systems.

Dataset Features

The dataset contains **46** columns, described as follows:

1. **id:** Unique identifier of the accident record.
2. **source:** Source of the accident report (e.g., MapQuest, Bing).
3. **severity:** Severity of the accident (target variable, 1 to 4).
4. **start_time:** Start time of the accident in local time zone.
5. **end_time:** End time of the accident in local time zone.
6. **start_lat:** Latitude in GPS coordinates of the start point.
7. **start_lng:** Longitude in GPS coordinates of the start point.
8. **end_lat:** Latitude in GPS coordinates of the end point (if available).
9. **end_lng:** Longitude in GPS coordinates of the end point (if available).

10. **distance_mi**: The length of the road extent affected by the accident in miles.
11. **description**: Natural language description of the accident.
12. **street**: Street name in the address field.
13. **city**: City in the address field.
14. **county**: County in the address field.
15. **state**: State in the address field.
16. **zipcode**: Zipcode in the address field.
17. **country**: Country in the address field (US).
18. **timezone**: Timezone based on the location of the accident.
19. **airport_code**: Denotes an airport-based weather station closest to the location.
20. **weather_timestamp**: Time-stamp of weather observation record (in local time).
21. **temperature_f**: Temperature (in Fahrenheit).
22. **wind_chill_f**: Wind chill (in Fahrenheit).
23. **humidity_pct**: Humidity (in percentage).
24. **pressure_in**: Air pressure (in inches).
25. **visibility_mi**: Visibility (in miles).
26. **wind_direction**: Wind direction.
27. **wind_speed_mph**: Wind speed (in miles per hour).
28. **precipitation_in**: Precipitation amount in inches.
29. **weather_condition**: Weather condition description (e.g., Rain, Snow, Clear).
30. **amenity**: Boolean flag indicating presence of an amenity nearby.
31. **bump**: Boolean flag indicating presence of a speed bump nearby.
32. **crossing**: Boolean flag indicating presence of a crossing nearby.
33. **give_way**: Boolean flag indicating presence of a give-way sign nearby.
34. **junction**: Boolean flag indicating presence of a junction nearby.
35. **no_exit**: Boolean flag indicating presence of a no-exit sign nearby.
36. **railway**: Boolean flag indicating presence of a railway nearby.
37. **roundabout**: Boolean flag indicating presence of a roundabout nearby.
38. **station**: Boolean flag indicating presence of a station (e.g., bus, train) nearby.
39. **stop**: Boolean flag indicating presence of a stop sign nearby.
40. **traffic_calming**: Boolean flag indicating presence of traffic calming means nearby.
41. **traffic_signal**: Boolean flag indicating presence of a traffic signal nearby.
42. **turning_loop**: Boolean flag indicating presence of a turning loop nearby.
43. **sunrise_sunset**: Indicates day or night based on sunrise/sunset.

- 44. **civil_twilight:** Indicates day or night based on civil twilight.
- 45. **nautical_twilight:** Indicates day or night based on nautical twilight.
- 46. **astronomical_twilight:** Indicates day or night based on astronomical twilight.

Target Variable

For the purposes of this project, the key variable of interest is **severity**, which serves as the multi-class label for classification. The goal is to predict this variable based on weather, time, and road characteristics. In this dataset, severity is defined as the impact of the accident on traffic flow, measured on a scale from 1 to 4:

- **1:** Short delay, minimal impact on traffic.
- **2:** Moderate impact, often involving lane closures.
- **3:** High impact, significant traffic congestion.
- **4:** Long-term delay, potentially involving total road closure.

3 Architecture of Data Pipeline

This project follows an end-to-end Big Data architecture to handle, process, and analyze the large-scale US Accidents dataset. The architecture consists of several well-defined stages, with clear input and output formats at each step:

1. Data Ingestion (Python → PostgreSQL → HDFS):

- **Input:** Raw CSV file `US_Accidents_March23.csv` (approx. 7.7 million rows).
- **Process:** Processed via Python in chunks of 200,000 rows for deduplication, missing value handling, and coordinate validation. Loaded into PostgreSQL tables using the efficient `COPY` command. Finally, exported to HDFS using Apache Sqoop.
- **Output:** Files in AVRO format (Snappy-compressed) with accompanying `.avsc` schema in HDFS.

AVRO was selected for HDFS storage to ensure robust schema preservation of data types during ingestion and to optimize write performance.

Snappy compression was applied due to its high processing speed and splittability, which facilitates efficient parallel processing across the Hadoop cluster while significantly reducing the disk footprint.

2. Storage and Query (Hive):

- **Input:** AVRO files and schema in HDFS.
- **Process:** External tables defined in Hive. To optimize analytical performance, data is then transferred into a final Hive table (`us_accidents_part_buck`) that is partitioned by **state** and bucketed by **severity** into 4 buckets. EDA is performed using HiveQL.
- **Output:** Queryable Hive tables stored in highly compressed ORC format, alongside several exported CSV files containing EDA query results.

3. Analysis and Modeling (PySpark):

- **Input:** Optimized Hive ORC tables accessed via Apache Spark.
- **Process:** Advanced feature engineering (including cyclical time transformation and Lat/Lng to ECEF 3D Cartesian coordinates), data scaling, and multi-class classification modeling (Random Forest and Logistic Regression) using Spark MLlib executed on a YARN cluster.
- **Output:** Trained ML models saved to HDFS, along with evaluation metrics and JSON/CSV prediction datasets.

4. Presentation (Dashboard):

- **Input:** Analytical EDA results (CSV) and model prediction outputs.
- **Process:** Dashboard design and storytelling to communicate findings.
- **Output:** Interactive dashboard providing visualizations and actionable insights into accident severities.

4 Data Preparation

ER Diagram

The relational model for this project is highly denormalized to optimize analytical queries. It consists of a single primary table `us_accidents` where the `id` column serves as the Primary Key. All temporal, spatial, weather, and road feature attributes are stored as columns within this entity.

us_accidents	
id	text
source	text
severity	smallint
start_time	timestamp without time zone
end_time	timestamp without time zone
start_lat	double precision
start_lng	double precision
end_lat	double precision
end_lng	double precision
distance_mi	double precision
description	text
street	text
city	text
county	text
state	character varying
zipcode	text
country	character varying
timezone	text
airport_code	text
weather_timestamp	timestamp without time zone
temperature_f	double precision
wind_chill_f	double precision
humidity_pct	double precision
pressure_in	double precision
visibility_mi	double precision
wind_direction	text
wind_speed_mph	double precision
precipitation_in	double precision
weather_condition	text
amenity	boolean
bump	boolean
crossing	boolean
give_way	boolean
junction	boolean
no_exit	boolean
railway	boolean
roundabout	boolean
station	boolean
stop	boolean
traffic_calming	boolean
traffic_signal	boolean
turning_loop	boolean
sunrise_sunset	text
civil_twilight	text
nautical_twilight	text
astronomical_twilight	text

Figure 1: ER Diagram of the dataset

Samples from the database

After bulk loading the cleaned CSV file using PostgreSQL's `COPY` command, we verified the integrity of the data. Below is a sample of the structured records.

Query Query History

```
1 SELECT * FROM us_accidents LIMIT 10;
```

Data Output Messages Notifications

	id [PK] text	source text	severity smallint	start_time timestamp without time zone	end_time timestamp without time zone	start_lat double precision	start_lng double precision	end_lat double precision	end_lng double precision	distance_mi double precision	de te
1	A-1	Source2	3	2016-02-08 05:46:00	2016-02-08 11:00:00	39.865147	-84.058723	[null]	[null]	0.01	Ri
2	A-2	Source2	2	2016-02-08 06:07:59	2016-02-08 06:37:59	39.928059000000001	-82.831184	[null]	[null]	0.01	Ac
3	A-3	Source2	2	2016-02-08 06:49:27	2016-02-08 07:19:27	39.063148	-84.032608	[null]	[null]	0.01	Ac
4	A-4	Source2	3	2016-02-08 07:23:34	2016-02-08 07:53:34	39.747753	-84.205581999999998	[null]	[null]	0.01	Ac
5	A-5	Source2	2	2016-02-08 07:39:07	2016-02-08 08:09:07	39.627781	-84.188354	[null]	[null]	0.01	Ac
6	A-6	Source2	3	2016-02-08 07:44:26	2016-02-08 08:14:26	40.100590000000004	-82.925193999999998	[null]	[null]	0.01	Ac
7	A-7	Source2	2	2016-02-08 07:59:35	2016-02-08 08:29:35	39.758274	-84.230506999999997	[null]	[null]	0	Ac
8	A-8	Source2	3	2016-02-08 07:59:58	2016-02-08 08:29:58	39.770382	-84.194901	[null]	[null]	0.01	Ac
9	A-9	Source2	2	2016-02-08 08:00:40	2016-02-08 08:30:40	39.778061	-84.172005	[null]	[null]	0	Ac
10	A-10	Source2	3	2016-02-08 08:10:04	2016-02-08 08:40:04	40.100590000000004	-82.925193999999998	[null]	[null]	0.01	Ri

Figure 2: Sample US Accidents Records from PostgreSQL

Data Preparation and Hive Table Creation

The initial data setup involves preparing HDFS and defining Hive structures. AVRO schemas (.avsc files) generated by Sqoop are first placed in a designated HDFS location. Then, within Hive, the `team31_projectdb` database is created.

To optimize performance, we used a two-step process. First, an external table mapped to the AVRO data was created. Second, the data was inserted into a highly optimized `us_accidents_part_buck` table, which is partitioned by `state` and bucketed by `severity` using the ORC format. Key HiveQL statements for this process include:

```
1 CREATE DATABASE IF NOT EXISTS team31_projectdb
2   LOCATION 'project/hive/warehouse';
3
4 USE team31_projectdb;
5
6 -- 1. Create External AVRO Table
7 CREATE EXTERNAL TABLE IF NOT EXISTS us_accidents
8   STORED AS AVRO
9   LOCATION 'project/warehouse/us_accidents'
10  TBLPROPERTIES ('avro.schema.url'='project/warehouse/avsc/us_accidents.avsc');
11
12 -- 2. Create Optimized ORC Table (Partitioned & Bucketed)
13 CREATE EXTERNAL TABLE IF NOT EXISTS us_accidents_part_buck (
14   id STRING, source STRING, severity INT, start_time TIMESTAMP,
15   -- [other columns omitted for brevity] --
16   astronomical_twilight STRING
17 )
18 PARTITIONED BY (state STRING)
19 CLUSTERED BY (severity) INTO 4 BUCKETS
20 STORED AS ORC
21 LOCATION 'project/hive/warehouse/us_accidents_part_buck'
22 TBLPROPERTIES ('orc.compress'='NONE');
```

Script Execution Workflow

The entire data ingestion and processing pipeline is orchestrated via bash scripts. These scripts handle Sqoop imports, upload AVRO schemas to HDFS, and execute a sequence of HiveQL scripts using `beeline`. Example commands from our execution workflow are:

```
1 # Uploading AVSC schema file to HDFS
2 hdfs dfs -mkdir -p "project/warehouse/avsc"
3 hdfs dfs -put "output/sqoop_generated/us_accidents.avsc" \
4   "project/warehouse/avsc/us_accidents.avsc"
```

```

5
6 # Executing HiveQL scripts via Beeline
7 beeline -u "jdbc:hive2://hadoop-03.uni.innopolis.ru:10001" -n "team31" \
8 -p "$HIVE_PASSWORD" --hiveconf "DB_NAME=team31_projectdb" \
9 -f hive/create_external_table.hql

```

5 Data Analysis

Exploratory Data Analysis (EDA) was performed using HiveQL. The queries aggregated the 7.7 million records across multiple dimensions including time, location, weather, and road infrastructure. The key findings were visualized to identify patterns that lead to traffic accidents and to understand the distribution of our target variable.

Analysis Results. Main insights

- **Target Variable Imbalance (Severity Distribution)**

The analysis of the `severity` column reveals a massive class imbalance in the dataset. Nearly 80% (79.67%) of all accidents are classified as Severity 2 (moderate impact, such as a lane closure). Severity 3 accounts for 16.81%, while extreme cases—Severity 4 (total road closure) and Severity 1 (minimal impact)—make up only 2.65% and 0.87% respectively. This baseline metric is crucial as it dictates the necessity of using the F1-score rather than raw accuracy during the Machine Learning evaluation phase.

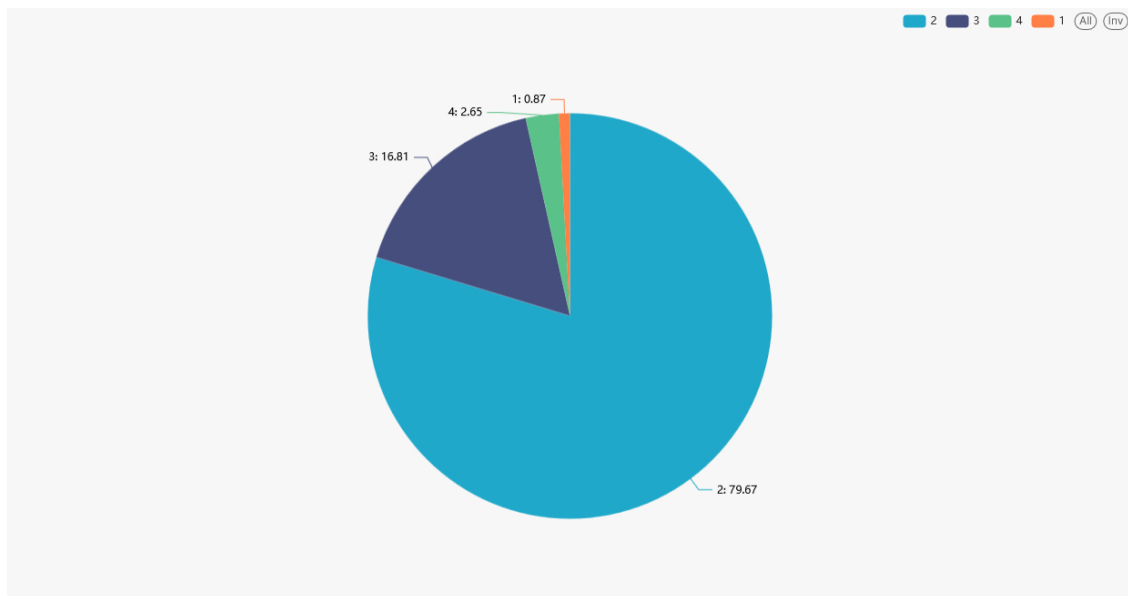


Figure 3: Insight 1: Overall Accident Severity Distribution

- **Temporal Patterns (Accidents by Hour of Day)**

The data shows a clear, classic bi-modal distribution corresponding to daily commuting patterns. Accidents peak sharply during the morning rush hour (7:00 AM – 9:00 AM), reaching up to 587k incidents at 7:00 AM. A second, equally massive peak occurs during the evening commute (4:00 PM – 6:00 PM), peaking at 582k incidents at 5:00 PM. This confirms that traffic volume is the absolute primary driver of accident frequency.

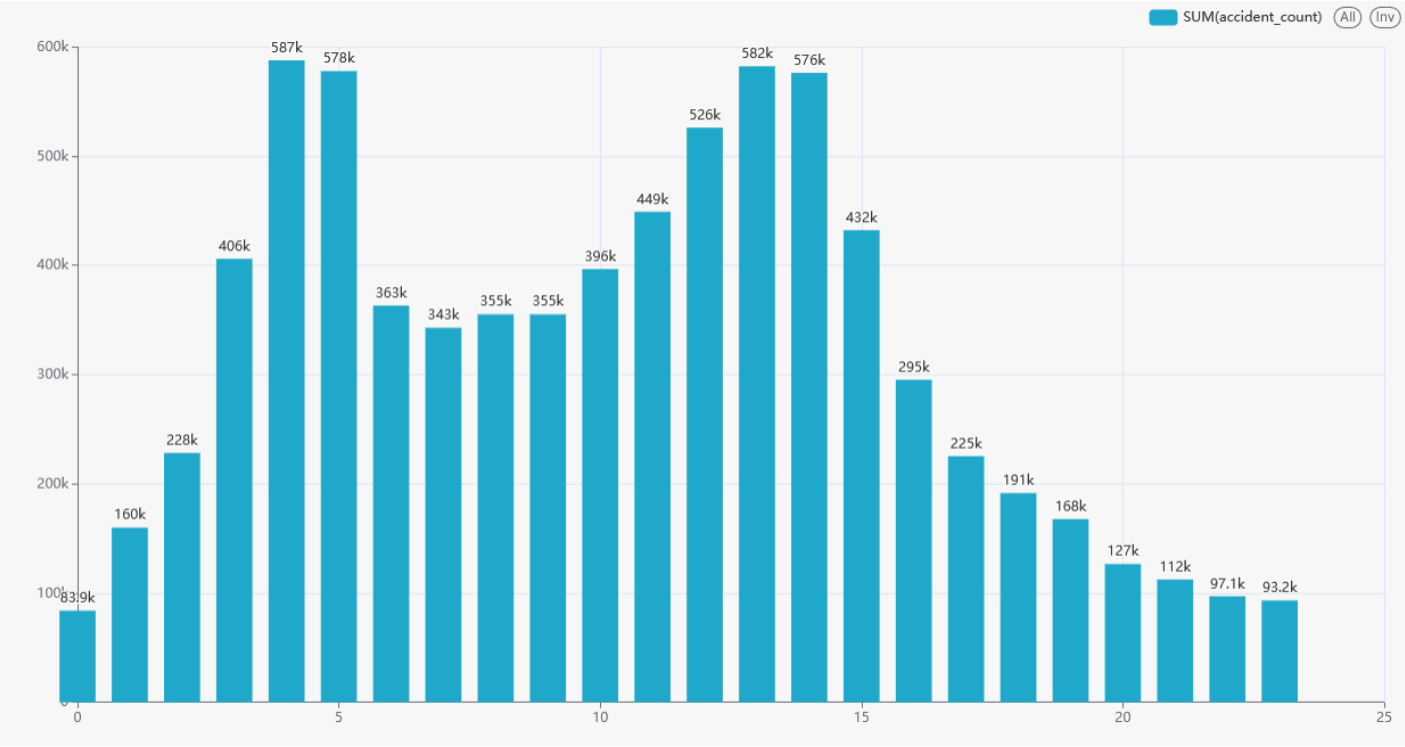


Figure 4: Insight 2: Accident Frequency by Hour of the Day

• Impact of Road Features

Analyzing boolean road features reveals where accidents are most likely to occur in relation to infrastructure. Junctions (20.23%) and Give Way signs (12.20%) are associated with the highest number of infrastructure-related accidents. This aligns with traffic dynamics, as junctions require lane changes, merging, and crossing traffic flows, inherently increasing collision risks. Conversely, Roundabouts account for only 3.34%, suggesting they might be a safer intersection design.

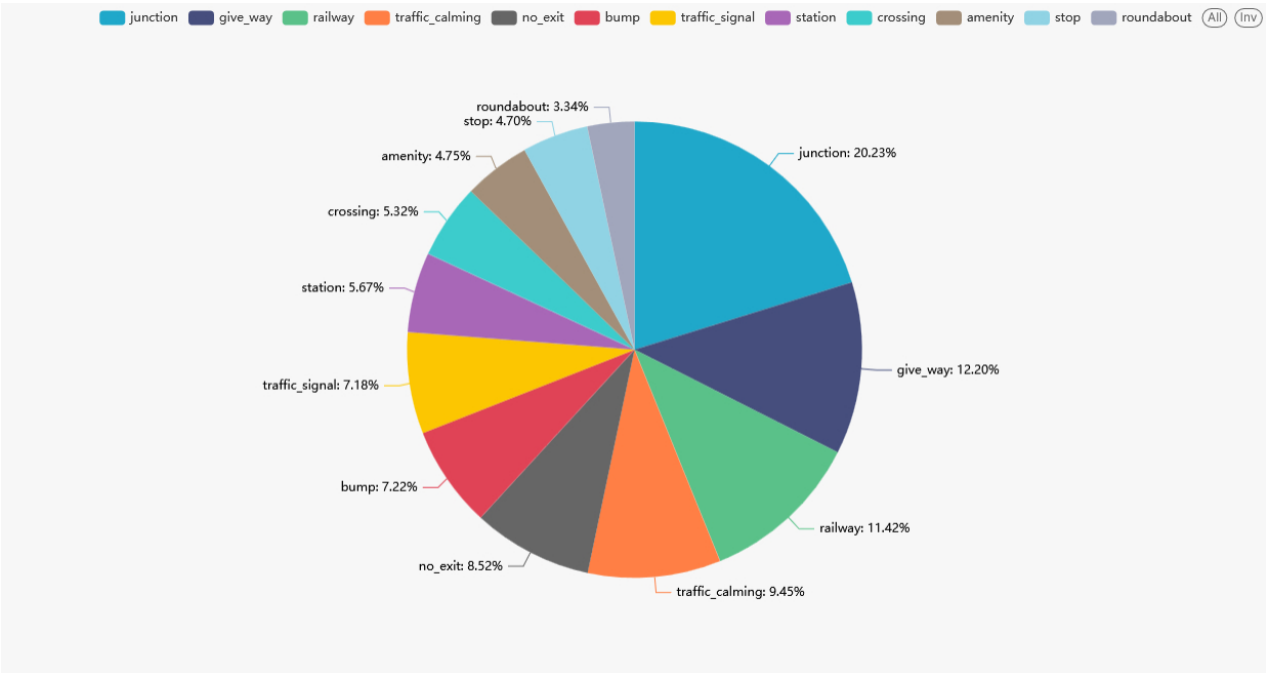


Figure 5: Insight 3: Proportion of Accidents Near Specific Road Features

• Geographical Hotspots (Top Cities)

When grouping accidents by city, Miami leads significantly with 187k recorded accidents, followed by Houston (169k) and Los Angeles (156k). These cities are characterized by massive urban sprawl, high population density, and heavy reliance on personal vehicles and highway networks, making them prime hotspots for traffic incidents.

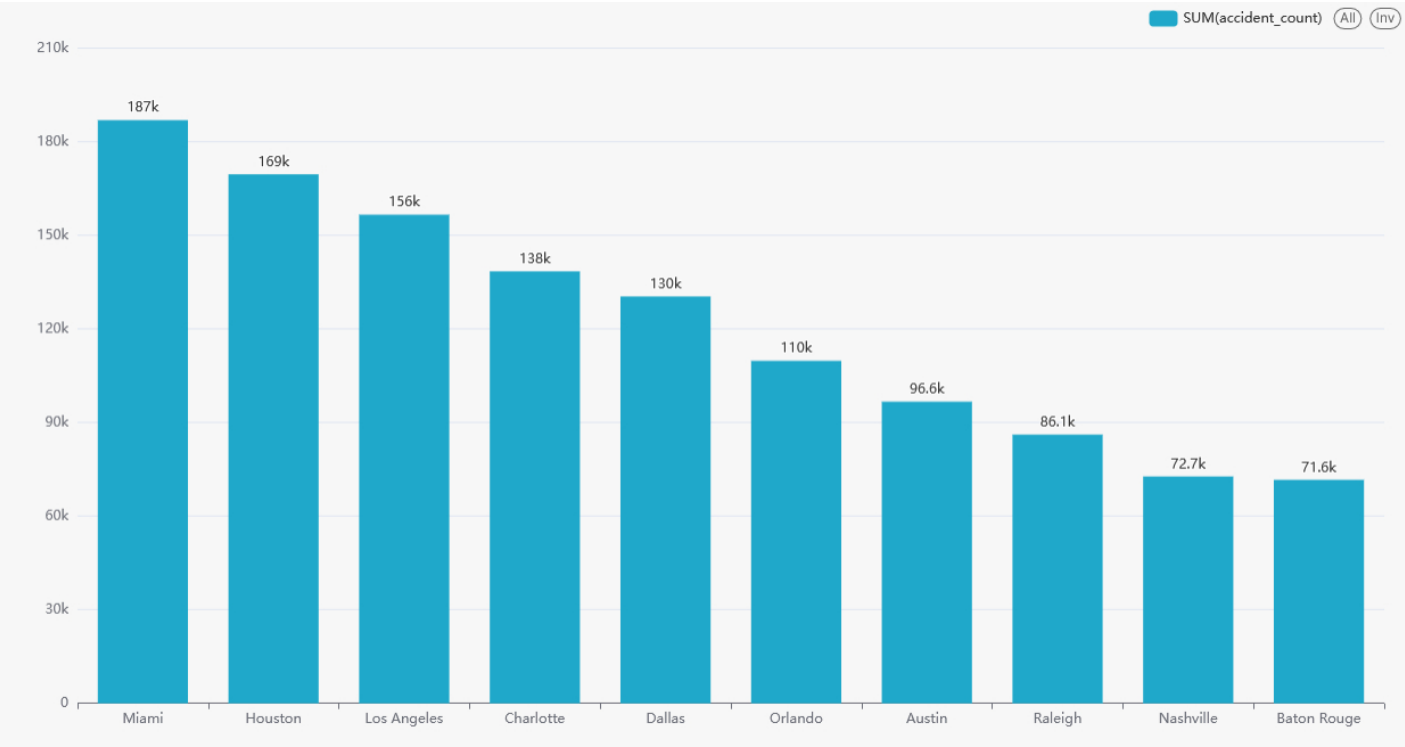


Figure 6: Insight 4: Top 10 Cities with Highest Accident Counts

• Weather Conditions Overview

The analysis of weather conditions indicates a wide diversity of atmospheric states during accidents. While many accidents happen under clear or mostly cloudy skies (simply because these are the most common weather states), a significant portion of incidents is distributed across adverse conditions like Scattered Clouds, Overcast, Rain, and Snow.

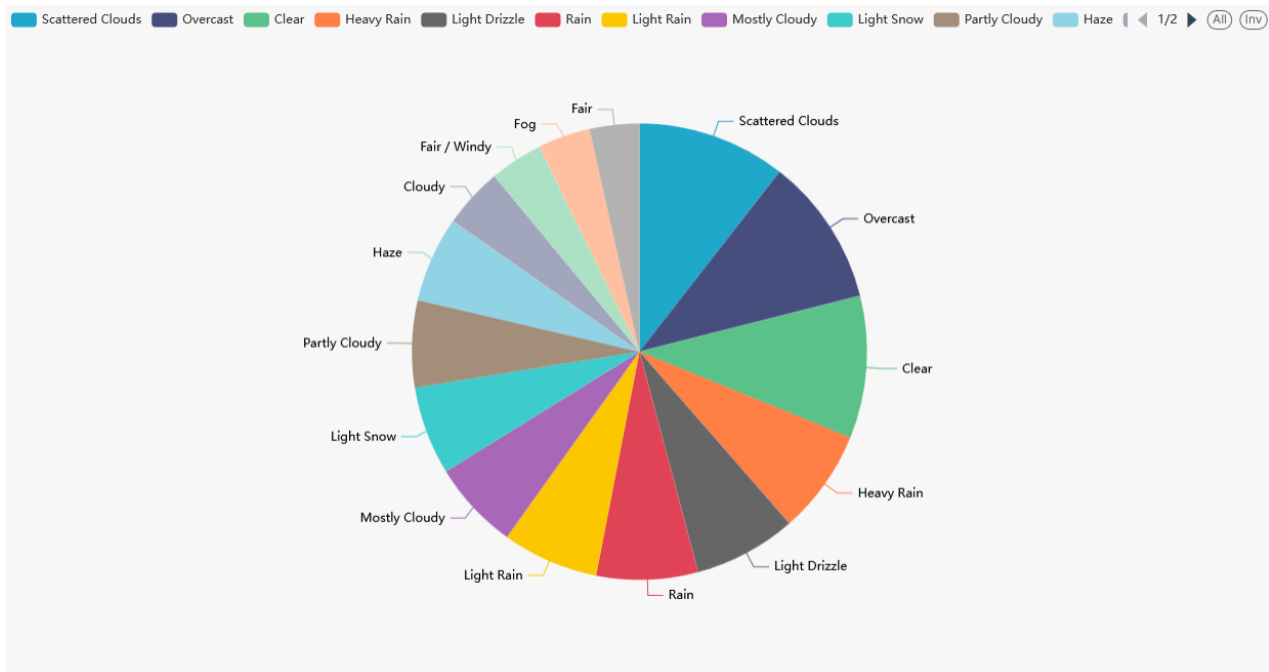


Figure 7: Insight 5: Diversity of Weather Conditions During Accidents

• The Paradox of Visibility and Wind Speed on Severity

An intriguing insight emerges when correlating high-severity accidents (Severity 3 and 4) with visibility and wind speed:

- **Visibility:** High severity peaks at moderate visibility (1-7 miles) at ~21%. Surprisingly, near-zero visibility (0-1 miles) has the lowest high-severity rate (17.58%).
- **Wind Speed:** Similarly, high severity peaks during moderate-to-strong winds (15-35 mph). At extreme wind speeds (35+ mph), the severity rate drops to 18.03%.

Interpretation: This counter-intuitive trend suggests a behavioral shift. In absolutely extreme conditions (zero visibility, hurricane winds), drivers either stay home or drive extremely slowly, resulting in minor fender-benders. In moderate-to-bad conditions, drivers often maintain highway speeds, leading to much more severe impacts when collisions occur.

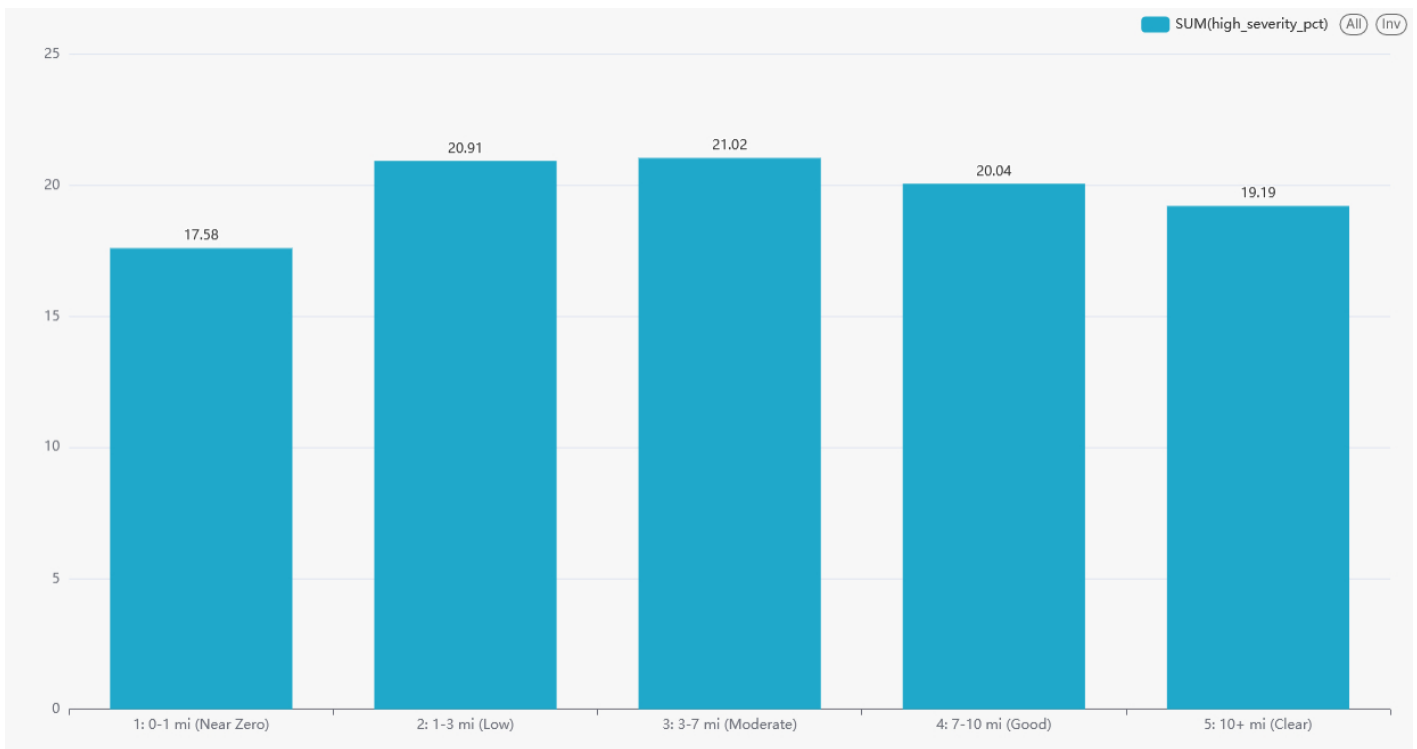


Figure 8: Insight 6: Percentage of High Severity Accidents by Visibility Range

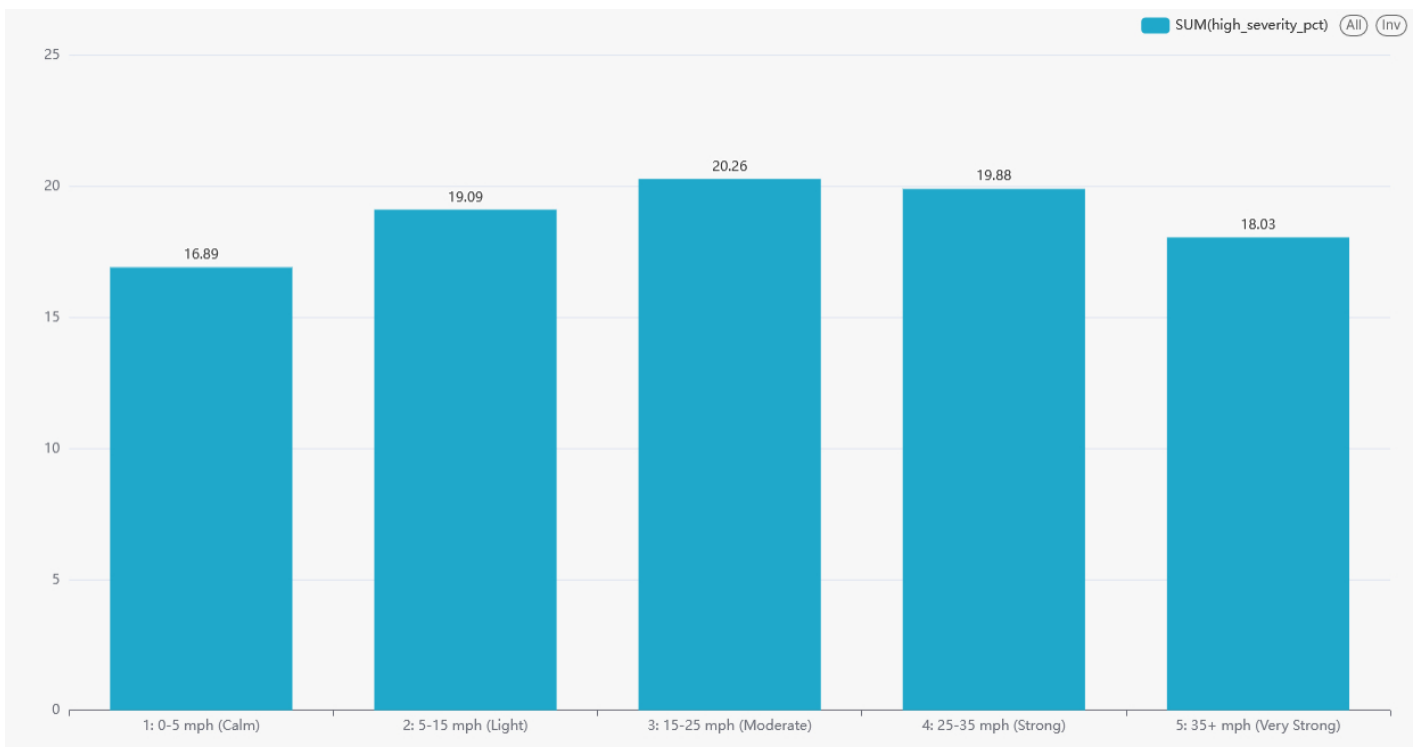


Figure 9: Insight 7: Percentage of High Severity Accidents by Wind Speed Range

6 Modeling and Prediction

The primary objective of this stage was to develop, train, and evaluate machine learning models using PySpark MLlib to predict the severity of traffic accidents. The process involved comprehensive data preparation, advanced feature engineering, model selection, hyperparameter tuning, and evaluation, all executed on a distributed YARN

cluster leveraging data stored in Hive and HDFS.

Dataset overview

- **Source:** Data was extracted from the `team31_projectdb` database, from an optimized Hive table named `us_accidents_part_buck`.
- **Target variable:** The multi-class column `severity` (values 1 to 4). For Spark ML compatibility, this was shifted to a 0-to-3 index (`label = severity - 1.0`).
- **Initial size:** The raw dataset contained **7,728,394 records**.

Data preparation pipeline

A robust preprocessing pipeline was constructed using PySpark to prepare the data for multi-class classification modeling:

1. **Feature selection and missing values handling:** A total of **41 features** were selected, including temporal, environmental, and road infrastructure variables. Rows with null values in essential columns (`severity`, `start_time`, `start_lat`, `start_lng`) were filtered out. For other columns, missing categorical values were replaced with `"unknown"`, missing numericals with `0.0`, and missing booleans with `False`.
2. **Advanced Time features decomposition:** Raw timestamps were replaced by a custom transformer called `CyclicalTimestampTransformer`. Temporal components (month, day, hour) were transformed using sine and cosine functions to accurately capture their cyclical nature (e.g., ensuring 23:00 and 01:00 are recognized as being close in time).
3. **Geospatial transformation:** A custom `LatLngToEcefTransformer` converted raw GPS coordinates (Latitude/Longitude) into 3D Cartesian coordinates (`ecef_x`, `ecef_y`, `ecef_z`) to provide a better spatial representation for the decision trees.
4. **Categorical features encoding:** String features such as `state`, `wind_direction`, and `weather_condition` were converted into numerical indices using `StringIndexer`.
5. **Feature assembling and scaling:** All 41 processed features (booleans, cyclical time, ECEF coordinates, scaled numericals, and indexed categoricals) were combined into a single feature vector column using `VectorAssembler`. The assembled vector was then standardized using `StandardScaler`.
6. **Train-test split:** The dataset was randomly split into training (60%) and testing (40%) sets using a fixed seed for reproducibility.

ML model training with hyperparameters tuning

Two classification models were trained and tuned using 3-fold cross-validation on the training data. The `MulticlassClassificationEvaluator` with the weighted F1-score metric was used to guide hyperparameter selection, which is critical due to the massive class imbalance (Severity 2 dominating).

- **Multinomial Logistic Regression**
 - Hyperparameters tuned via `ParamGrid`: `regParam` ($[10^{-3}, 10^{-2}, 0.05]$), `elasticNetParam` ($[0.0, 0.5]$).
- **Random Forest Classifier**
 - Hyperparameters tuned via `ParamGrid`: `numTrees` ($[40, 80, 120]$), `maxDepth` ($[6, 10]$).
 - Best parameters found: `numTrees` = 40.

Model evaluation and comparison

Models were evaluated on the 40% test set using Accuracy and the weighted F1-score.

Model	numClasses	numFeatures	Accuracy	F1-score
Logistic Regression	4	41	79.95%	76.19%
Random Forest	4	41	81.86%	76.72%

Table 1: Model Comparison Metrics on Test Set

As shown in Table 1, the Random Forest classifier demonstrated superior performance across all evaluated metrics. It successfully captured the non-linear relationships between weather conditions, road infrastructure, and time, achieving the highest Accuracy (81.86%) and F1-score (76.72%).

Model and prediction outputs

- **Models saved:** The best trained Logistic Regression and Random Forest models were serialized and saved to HDFS (in the `project/models/` directory).
- **Predictions saved:** Predictions made by both models on the test set, containing the actual `label` and the `prediction`, were saved to HDFS as JSON and CSV files.
- **Evaluation results saved:** The comparison metrics were saved to HDFS as `evaluation.csv` for integration into the final visualization dashboard.

7 Data presentation

To present our findings, we developed a multi-tab interactive dashboard in Apache Superset. The dashboard is publicly available at [link](#).

It consists of three main tabs:

Tab 1: Data Description

This tab provides a high-level summary of the raw data characteristics.

- **Big Number Indicators:** Showcases the 7.7M records, 46 features, and 49 states coverage.
- **Data Dictionary:** Explains the datatypes (Double, String, Boolean) and the Stage 1 cleaning process.
- **Dataset Sample:** A live table previewing the first 10 records.

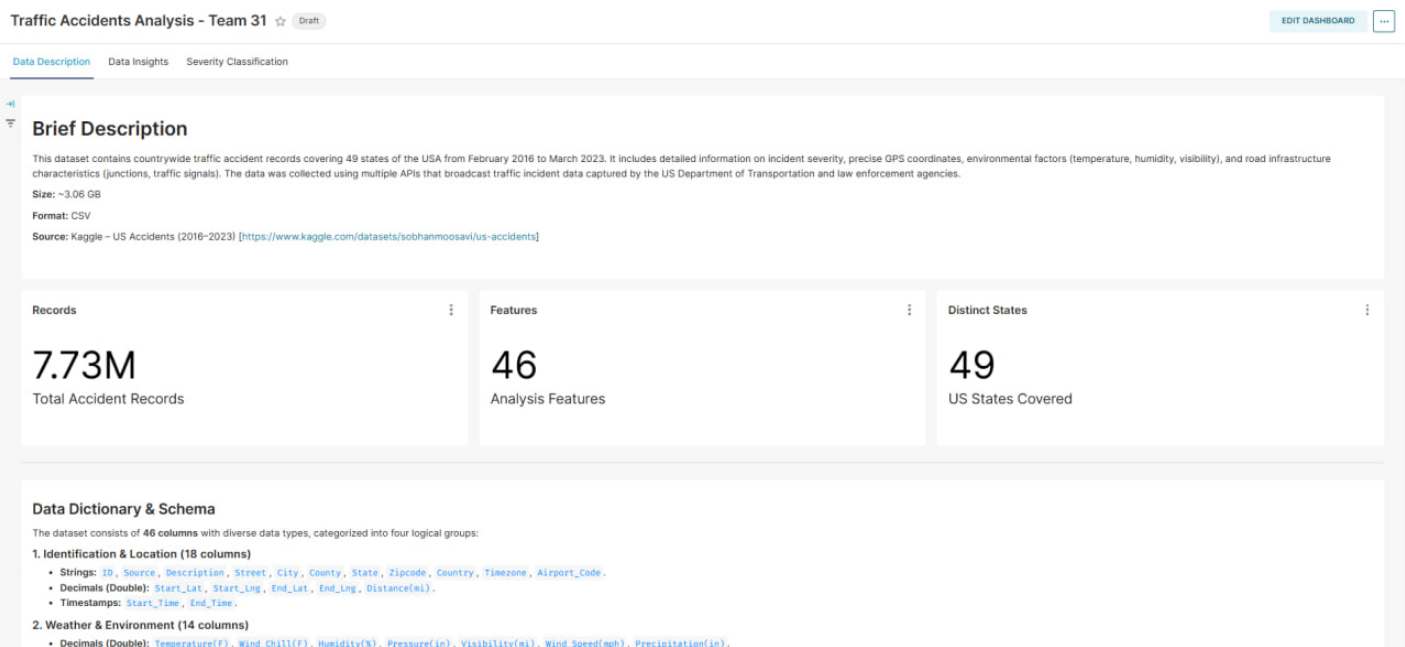


Figure 10: Dashboard Tab 1 – Data Overview

Tab 2: Data Insights (EDA)

Includes seven visual charts supporting the exploratory data analysis, such as overall accident severity distribution, hourly frequency trends, geographical hotspots by city, and the correlation between incident severity and environmental factors like visibility, wind speed, and road infrastructure characteristics. Each insight is also documented and interpreted in detail in Data Analysis Section of the report.

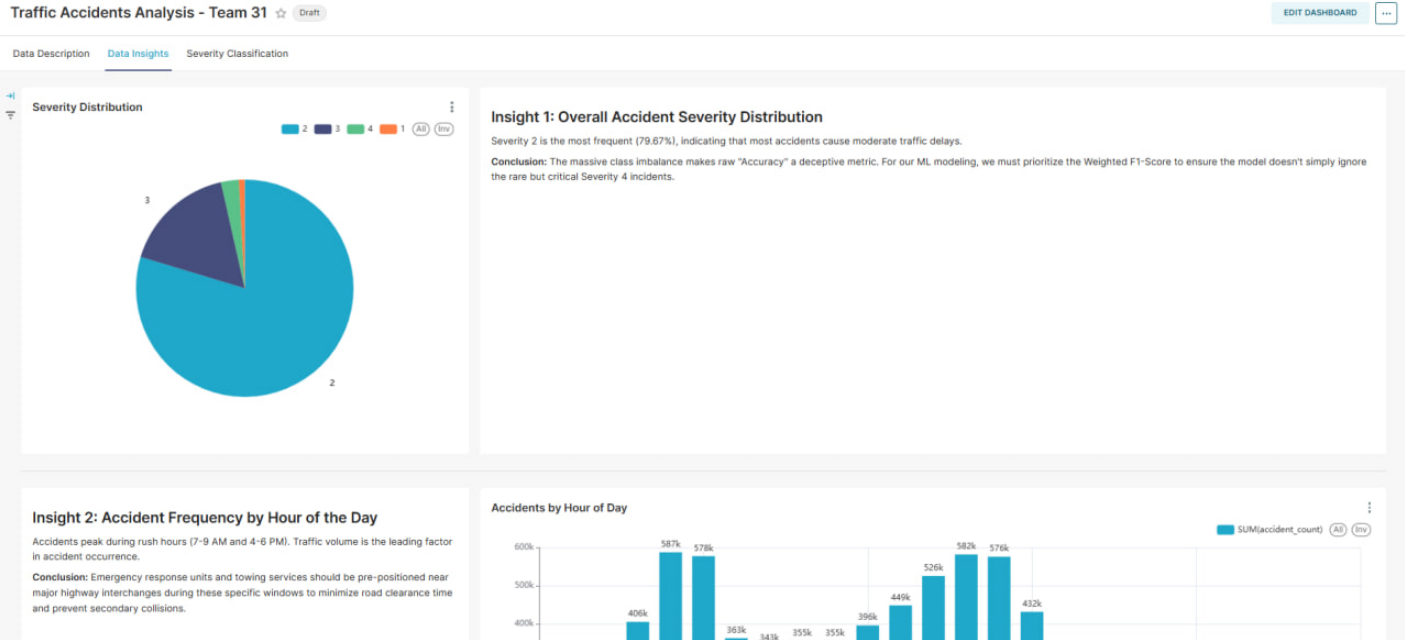


Figure 11: Dashboard Tab 2 – Exploratory Data Analysis

Tab 3: Severity Prediction

This tab presents the final stage of the project — model evaluation and multi-class prediction of accident severity. It includes:

- **Overview panel:** Summarizes the goal of building a multi-class classifier using PySpark MLlib to predict the target variable `severity`. It highlights the dataset distribution of approximately 7.7 million records, noting the significant class imbalance where Severity 2 accounts for 79.67% of all incidents.
- **Model training summary:** Outlines the distributed training and hyperparameter tuning process performed on the YARN cluster. For each model, the grid search configuration and the best selected parameters are listed (e.g., `regParam = 0.01` and `elasticNetParam = 0.0` for Logistic Regression; `numTrees = 40` and `maxDepth = 10` for Random Forest).
- **Performance metrics:** Evaluation results for the best models are displayed using Accuracy and Weighted F1-score. As detailed in the ML Modeling section, the Random Forest model achieved the highest performance with an accuracy of 81.86% and an F1-score of 76.72%, consistently outperforming the Logistic Regression baseline.
- **Prediction examples:** Two interactive tables visualize live prediction samples from the test set. These tables compare the **Actual** labels versus the **Predicted** severity levels (indexed 0–3), allowing users to observe the model’s classification tendencies.

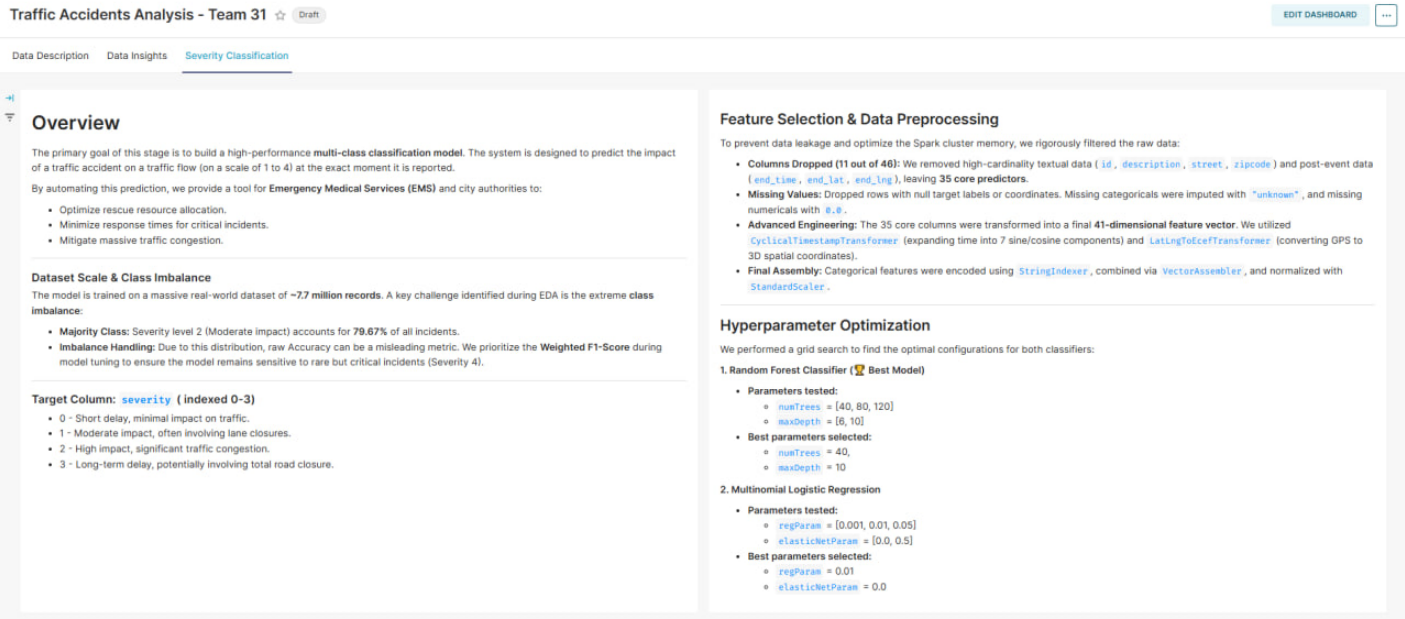


Figure 12: Model overview and training settings

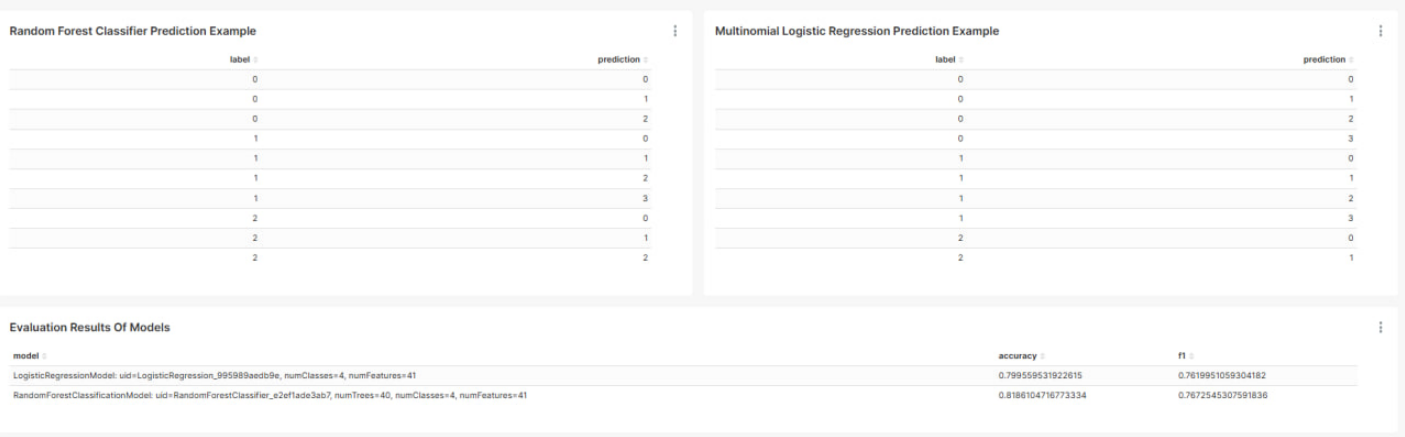


Figure 13: Evaluation results and example predictions for both models

Findings

The most critical finding is that accident severity is not entirely random; it is highly correlated with geographical location (ECEF coordinates) and specific temporal periods. While road features like traffic signals reduce minor accidents, severe accidents (Level 3 and 4) are heavily influenced by a combination of high-speed roads and sudden drops in visibility or adverse weather.

8 Conclusion

Summary of the report

In this project, we successfully developed and deployed an end-to-end Big Data pipeline for predicting traffic accident severity. The key achievements include:

- Successfully handled a massive dataset of 7.7 million records, implementing a robust ETL process from raw CSV to a distributed Hadoop ecosystem.
- Leveraged PostgreSQL for initial staging and Apache Hive for optimized analytical warehousing.
- Utilized Apache Spark MLlib to build a predictive pipeline incorporating custom transformations.
- Developed a Random Forest classifier that achieved an accuracy of 81.86% and F1-Score 76.72%.
- Built Apache Superset dashboard.

9 Reflections on own work

Challenges and difficulties

During the development process, we faced several difficulties:

- Memory Management
- Data Imbalance
- Working in cluster in the first time
- Superset is not intuitively understandable
- Cluster is down several times

Recommendations

For future iterations of this project, we recommend the following enhancements:

- **Real-Time Ingestion:** Integrate Apache Kafka to allow the model to ingest streaming traffic data and provide instantaneous severity predictions.
- **Imbalance Mitigation:** Implement advanced techniques such as SMOTE or class-weighting within Spark to improve the recall of critical Severity 4 accidents.
- **Feature Expansion:** Incorporate external data sources such as high-resolution road surface sensors and real-time traffic flow density to increase predictive precision.

The table of contributions of each team member

Project tasks	Task description	Anna B.	Sofia P.	Daria A.	Aisylu F.	Deliverables	Hrs
Stage I: Ingestion	Data collection (Kaggle API), Python chunked cleaning, and Sqoop import to HDFS.	100%	0%	0%	0%	preprocess_data.py, import_to_hdfs.sh, stage1.sh, build_projectdb.py, data_collection.sh, data_storage.sh, preprocess.sh, sql/*.sql	12
Stage II: Storage	Hive warehouse design, partitioning by state, bucketing, and HiveQL analytics (Q1–Q7).	0%	100%	0%	0%	stage2.sh, hive/*.hql, output/q*.csv	14
Stage III: Spark ML	Distributed ML pipeline, ECEF/Cyclical features, Random Forest/LR training.	0%	0%	100%	0%	train_ml.py, models/, evaluation.csv, stage3.sh	18
Stage IV: Presentation	Superset Dashboard design, storytelling, and Tab 1–3 visual insights.	0%	0%	0%	100%	Published Dashboard, Tab screenshots	10
Automation	Development of stage4.sh for integration between Spark and Superset.	0%	0%	0%	100%	stage4.sh, report/	4
Reporting	Technical documentation, LaTeX formatting, and pipeline summary.	0%	0%	0%	100%	report.pdf	15

10 References

- Our Github Repository
- Kaggle Dataset: US Accidents (2016 - 2023)